Практикум 1. От вопроса к агенту

Практический материал лекций 1 и 2

От интеллектуального поиска к интеллектуальным агентам

О. М. Атаева А. П. Халов

ВМК МГУ, осень 2026

AI4Science AI4Research AI4Math

Что мы делаем и зачем

Проблема

Исследователь задаёт вопрос. Языковая модель отвечает уверенно, и проверить её ответ нечем: источников нет, вычислений нет, правил нет.

Что строим в курсе

Ассистента, который отвечает на вопросы по **вашей** предметной области и у каждого утверждения показывает, откуда оно взято и чем проверено. Модель в нём только одна из частей.

Что делаем сегодня

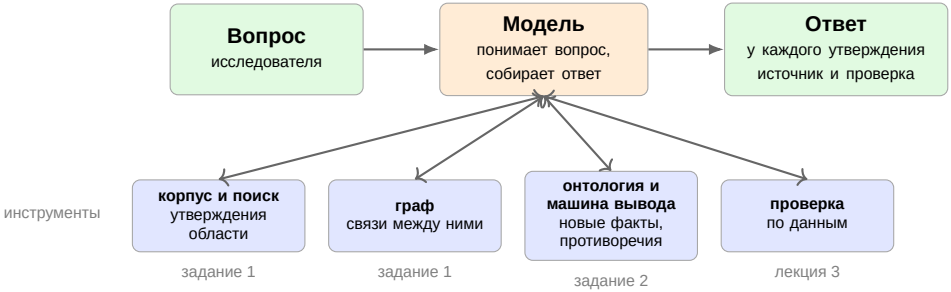
Собираем такого ассистента целиком на одном маленьком примере. Затем делаем первые два шага вашего проекта: корпус с графом и онтологию. Это и есть задания 1 и 2.

- Пример: редактор сборника решает, какие статьи выносить на редколлегию. В нём есть всё, что есть в любой предметной области: утверждения, связи между ними, данные для проверки.

Главное

К концу занятия виден весь путь от вопроса до проверенного ответа и ясно, какие два его шага делаются в заданиях.

Что строим в курсе



- Модель не хранит знание области и не проверяет факты. Это делают инструменты, которые строятся в заданиях над своей областью.
- Каждый инструмент появляется в своём примере и снимает одну ошибку модели.

Что собираем сегодня

Ассистент редактора сборника. На входе вопрос и три файла: журнал рецензий, положение о статьях, утверждения статьи P2. На выходе ответ, у каждой части которого стоит пометка: откуда взято и чем проверено.

Из лекции 1: чего не хватает модели

корпус положения, поиск по словам	пример 2
граф зависимостей, путь по нему	пример 2
правила над журналом	пример 3
внешняя проверка утверждений	пример 3
агент: модель просит, программа исполняет	пример 4

Из лекции 2: знание как данные

онтология: классы, свойства, индивиды, правило 6.2	пример 5
машина вывода: новые факты, противоречия	пример 5
агент, который проверяет данные до ответа	пример 6

В задании

Те же части для **вашей** области: корпус и граф в задании 1, онтология и машина вывода в задании 2. Оба задания сегодня делаются вживую на маленьком примере.

Данные задачи: три файла

Всё, что есть у редактора, лежит в трёх таблицах в формате CSV. Папка `data/`.

Файл	Что это	Столбцы
`reviews.csv`	журнал рецензий: 14 строк, по одной на рецензию, 8 статей	статья, организация авторов, рецензент, организация рецензента, дата назначения, дата сдачи, оценка, шкала, ответ авторов
`corpus_Aprime_standards.csv`	положение о статьях: 31 запись. 6 разделов, 11 терминов, 14 требований	id, тип, название, формулировка, зависит_от, источник
`paper_P2_claims.csv`	три утверждения статьи P2 с доказательствами	id, вид, формулировка, опирается_на, статус в статье, замечание рецензента, формальная постановка, вердикт внешней проверки

- Журнал есть **данные**: что произошло с каждой рецензией.
- Положение есть **знание**: требования, термины и связи между ними.
- Статья P2 есть **утверждения**, которые проверяются вне журнала и вне положения.

Главное

Корпус из задания 1 имеет те же шесть столбцов, что и положение. Данные для проверки в своей области могут иметь любой вид.

Как выглядят данные

Журнал, файл `data/reviews.csv`, формат CSV. Строки, на которых держится пример: оценка по шкале 5, рецензент из организации авторов, сдача через 27 дней, рецензия не сдана.

```
статья,авторы_организация,рецензент,рецензент_организация,назначено,сдано,оценка,шкала,ответ_авторов
P2,МФТИ,Rv3,ИТМО,2026-09-01,2026-09-12,4,5,да
P3,ФИЦ ИУ,Rv4,ФИЦ ИУ,2026-09-02,2026-09-16,9,10,да
P5,МГУ,Rv3,ИТМО,2026-09-03,2026-09-30,7,10,да
P8,ФИЦ ИУ,Rv5,МГУ,2026-09-04,,,,
```

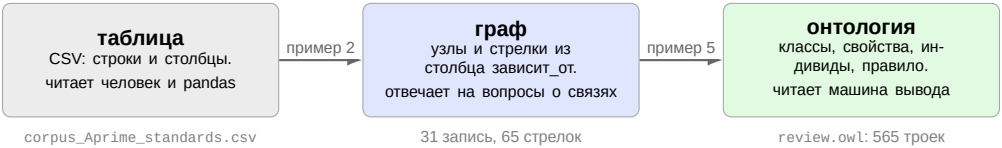
Положение, файл `data/corpus_Aprime_standards.csv`, формат CSV. Столбец `зависит_от`: прямые ссылки через точку с запятой.

```
id,  тип,         название,                       зависит_от,       источник
S06, Раздел,      Раздел 6. Порядок рецензирования, S01;S03,        Положение о статьях §6
T06, Термин,      Рецензия,                       S03;T05,          Положение о статьях §3.6
R62, Требование, 6.2 Независимость рецензента,  S06;T05;T04,      Положение о статьях §6.2
R65, Требование, 6.5 Принятие решения,          S06;R61;R64;T10,  Положение о статьях §6.5
```

Статья P2, файл `data/paper_P2_claims.csv`, формат CSV. Вердикт получен внешней проверкой доказательства и записан в файл.

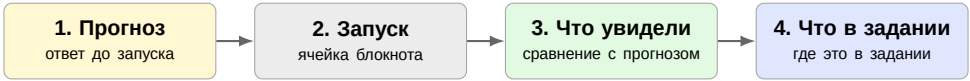
```
id, вид,          опирается_на, статус_в_статье, вердикт
C1, теорема,      A05;A14;A27,  доказано,        принято
C3, утверждение, A27,           доказано,        отклонено: доказательство не собирается; контрпример
```

Одни данные, три вида записи



- Содержание одно: положение о статьях и журнал. Меняется вид записи и то, какие вопросы по ней можно задать.
- Таблица отвечает «что записано». Граф отвечает «что с чем связано». Онтология отвечает «что из этого следует» и «нет ли противоречия».
- В задании 1 строится таблица и граф, в задании 2 онтология. Из одного и того же корпуса.

Правило занятия



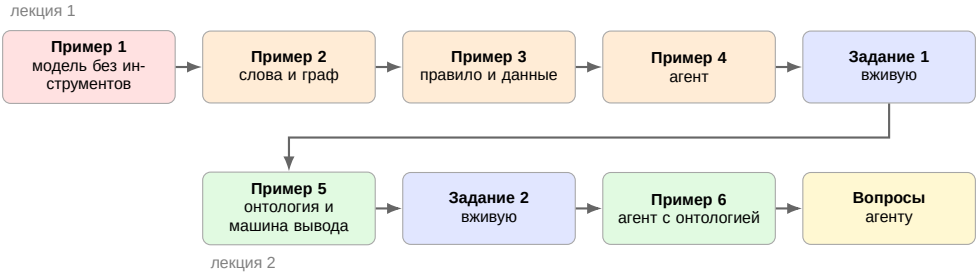
- Модель работает на этом ноутбуке через Ollama.
- Блокноты, код и данные входят в материалы занятия. Каждый пример повторяется одной ячейкой.

Шесть слов занятия

Слово	Что это	Где увидим
корпус	таблица утверждений области: что сказано, где сказано, на что опирается	пример 2, задание 1
граф	та же таблица, нарисованная стрелками: от утверждения к тому, на что оно опирается	пример 2, задание 1
правило	требование, записанное так, что его можно применить к данным	пример 3
онтология	запись области для машины: виды объектов, их связи и правила в одном файле	пример 5, задание 2
машина вывода	программа, которая получает из онтологии новые факты и находит противоречия	пример 5, задание 2
агент	модель, которой дали инструменты и правило отвечать только по их результатам	примеры 4 и 6

Точные определения даны на лекциях.

Карта занятия



- **Один вопрос** проходит через всё занятие. В начале на него отвечает модель, в конце агент.
- **Две области.** Примеры идут на задаче редактора из лекций. Задания показаны на новой маленькой области.

Пример 1. Модель без инструментов

Что ответит модель, если дать ей только текст?

Вопрос редактора

P1	P2	P3	P4	P5	P6	P7	P8
2 рец.	2 рец.	2 рец.	1 рец.	2 рец.	2 рец.	2 рец.	0 рец.

Какие статьи можно выносить на решение редколлегии, и на что опирается это требование?

Один вопрос на всё занятие: сначала отвечает модель, в конце агент.

Что есть у редактора

- 8 статей, журнал на 14 рецензий.
- Положение о статьях: 31 запись.

Четыре требования положения

- 6.1: не менее двух рецензий.
- 6.2: рецензент не из организации авторов.
- 6.5: есть ответ авторов. 6.6: срок 21 день.

Спросим модель

Модель получает вопрос и журнал текстом. Больше ничего.

Прогноз

Сколько статей из восьми модель разрешит вынести на решение?

А. Три **Б.** Пять или шесть **В.** Все восемь

Блокнот `practice_1_editor.ipynb`, пример 1. В терминале: `python3 ask.py --plain "вопрос"`.

Найдите ошибки

Все восемь статей можно выносить на решение редколлегии. Средние оценки: P1 --- 7,5. P2 --- 6,5. P3 --- 7,5. P4 --- 8. P5 --- 6,5. P6 --- 4. P7 --- 8,5. P8 --- рецензия ожидается, статья может быть рассмотрена по имеющимся данным.
Требование о принятии решения опирается на п. 6.1 (две рецензии) и п. 6.4 (ответ авторов), а также на ГОСТ Р 7.0.113-2021 «Рецензирование научных статей. Общие требования», п. 5.4.
По статье P2: замечание рецензента Rv3 к утверждению 3 необоснованно, доказательство утверждения 3 корректно.

- Запись характерного ответа. Живая модель отвечает своими словами.
- В ответе четыре места, которым нельзя верить.

Четыре ошибки

Что сказано	В чём дело	Чего не хватило
средняя оценка P2 равна 6,5	усреднены 4 из 5 и 9 из 10	**схемы данных**: у числа нет шкалы
ГОСТ Р 7.0.113-2021	такого документа нет	**источника**
все восемь можно выносить	требования 6.1, 6.2, 6.5, 6.6 не проверены	**правил** и их применения
доказательство корректно	утверждение 3 ложно	**внешней проверки**

- Пункты 6.1 и 6.4 названы верно: они есть в положении.
- По тексту ответа верное от выдуманного отличить нельзя.

Главное

Модель не ошиблась в языке. Ей не дали ни шкалы, ни связей между положениями, ни правил, ни проверки. Дальше это добавляется по одному.

Пример 2. Слова и граф

Где лежит ответ на вопрос «на что это опирается»?

Спросим поиск

Вторая половина вопроса: **на что опирается требование 6.5?** Зададим её поиску по словам.

Прогноз

Что вернёт поиск по словам?

А. Цепочку положений, на которые опирается 6.5

Б. Сам пункт 6.5 и положения с похожими словами

В. Ничего

Запрос: «на какие положения опирается требование 6.5 о принятии решения редколлегией».

Что нашёл поиск

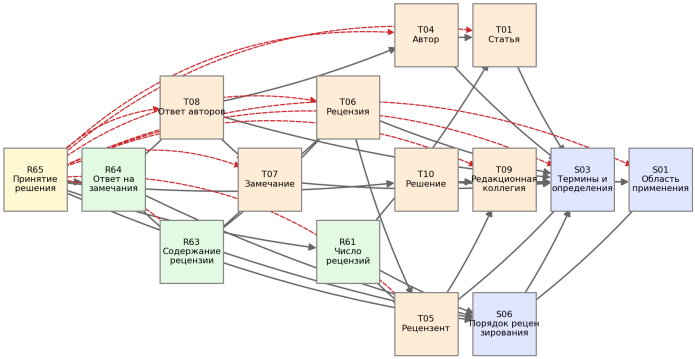
id	название	оценка
R65	**6.5 Принятие решения**	6,02
R53	5.3 Ключевые слова	3,01
R64	6.4 Ответ на замечания	2,81
R51	5.1 Объём статьи	2,73
R52	5.2 Аннотация	2,65

- Первым стоит сам пункт 6.5: в нём есть слова запроса.
- Дальше идут положения, где встретились слова «требование», «положение».
- Цепочки нет. В тексте пункта 6.5 нет слов «раздел 3» или «термин Рецензент».

Что увидели

Поиск по словам отвечает на вопрос «где об этом сказано». Ответ на вопрос «на что это опирается» записан в связях между документами.

Тот же вопрос графу



Жёлтый узел: требование 6.5. Сплошные стрелки записаны в корпусе. Красные пунктирные получены проходом по цепочкам.

Что увидели

Записано 4 зависимости, с учётом цепочек их 14. Каждый шаг цепочки есть в корпусе с источником.

Откуда взялся граф

Строки корпуса `corpus.csv`

```
id,  тип,        название,             зависит_от
R65, Требование, 6.5 Принятие решения, S06;R61;R64;T10
R64, Требование, 6.4 Ответ на замечания, S06;R63;T08
R63, Требование, 6.3 Содержание рецензии, S06;T06;T07
T07, Термин,     Замечание,            S03;T06
```

Столбец `зависит_от` превращает таблицу в граф: каждая ссылка становится стрелкой.

В задании

Задание 1. Файл `kg/corpus.csv` устроен так же: не меньше 30 записей, не меньше 10 с зависимостями.

Один из пяти вопросов исследователя должен быть вопросом о зависимости.

Главное

Граф хранит то, чего нет в тексте документа: его связи с другими документами.

Пример 3. Правило и данные

Как проверить статьи по положению?

Статья P2

статья	рецензент	оценка	шкала
P2	Rv3	**4**	**5**
P2	Rv1	**9**	**10**

Модель без инструментов назвала среднюю оценку 6,5.

Прогноз

Какая у P2 средняя оценка?

А. 6,5 **Б.** 8,5 **В.** 7,0



4 из 5 = 8 из 10

9 из 10

Вердикт по журналу

статья	выносить	оценка	нарушенное требование
P1	да	7,5	---
P2	да	**8,5**	---
P3	нет	7,5	6.2: рецензент из организации авторов
P4	нет	8,0	6.1: одна рецензия
P5	нет	6,5	6.6: рецензия сдана на 27-й день
P6	нет	4,0	6.5: нет ответа авторов
P7	да	8,5	---
P8	нет	---	6.1: рецензия не сдана

Что увидели

Готовы три статьи из восьми. У каждой из остальных назван пункт положения. Средняя оценка P2 равна 8,5.

Из чего состоит проверка

1. **Правило** из положения.
2. **Данные со схемой**: у оценки есть шкала, у даты есть отсчёт.
3. **Код**, который применяет правило к данным.

```
RULES = {
 "6.1": d["сдана"].sum() >= 2,
 "6.2": not d["своя_организация"].any(),
 "6.6": not d["просрочена"].any(),
 "6.5": (d["ответ_авторов"] == "да").all(),
}
```

В задании

Задание 1. Если в вашей области есть факт, который проверяется по данным, запишите одно правило по этому образцу.

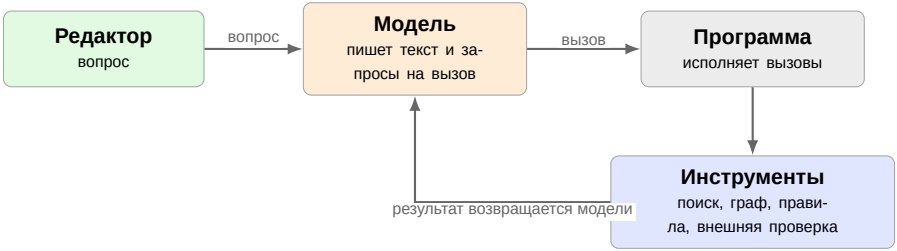
Вопрос типа «проверка» среди пяти вопросов исследователя.

Строка `"6.2"`: в примере 5 это требование уйдёт из кода в данные.

Пример 4. Агент

Что будет, если инструменты отдать модели?

Что такое агент



- Модель ничего не запускает сама. Она пишет: «вызови `check_rules`». Запускает программа.
- Результат возвращается модели текстом. Цикл идёт, пока модель не ответит без вызовов.
- В примерах 2 и 3 функции запускались вручную. Теперь их получает модель.

Что получает модель

Инструкция

Ты ассистент редактора сборника. Отвечай только по результатам инструментов. Каждую часть ответа помечай: [проверено: инструмент] или [найдено: инструмент]. Документы и пункты, которых нет в результатах инструментов, не называй. Если что-то не удалось подтвердить, пиши [не подтверждено].

Четыре инструмента

`search_words`	поиск по словам
`graph_deps`	на что опирается положение
`check_rules`	правила над журналом
`lean_check_claims`	внешняя проверка утверждений статьи P2

Прогноз

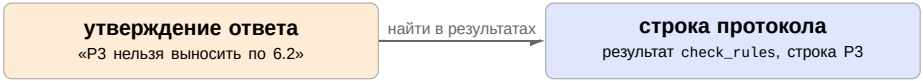
Какой инструмент модель вызовет первым? Сколько вызовов она сделает?

Протокол прогона

```
[1] модель вызвала check_rules({})
    <- [{"статья": "P1", "выносить": "да", "средняя оценка (шкала 10)": 7.5}, {"статья": "P2", ...
[2] модель вызвала search_words({'query': 'принятие решения редколлегией требование'})
    <- [{"id": "R65", "название": "6.5 Принятие решения", "bm25": 5.14}, ...
[3] модель вызвала graph_deps({'doc_id': 'R65'})
    <- {"документ": "R65", "опирается_на": ["R61 6.1 Число рецензий", "R63 6.3 Содержание ...
[4] модель вызвала lean_check_claims({})
    <- [{"id": "C1", "вердикт": "принято"}, {"id": "C2", "вердикт": "принято"}, {"id": "C3", ...
[5] ответ модели:
Можно выносить на решение: P1, P2, P7 [проверено: check_rules]. Нельзя: P3 по 6.2, P4 по 6.1 и 6.5,
P5 по 6.6, P6 по 6.5, P8 по 6.1 и 6.5 [проверено: check_rules]. Требование 6.5 (R65) опирается на
14 положений [найдено: graph_deps]. По статье P2: C1 и C2 приняты, C3 отклонено [проверено:
lean_check_claims]. Иных нормативных документов в результатах инструментов нет [не подтверждено].
```

Запись прогона. Результаты инструментов настоящие. У живой модели свой порядок вызовов и свои слова.

Как проверять ответ агента



- Каждое утверждение ответа ищется в результатах инструментов.
- Пометки в квадратных скобках подсказывают, где искать. Сами они проверкой не являются: их ставит модель.
- Утверждение, которого нет в результатах, порождено моделью и не проверено.

Главное

Агент сам ничего не проверяет. Он выбирает инструмент под часть вопроса и собирает ответ. Проверяют инструменты.

До и после

	Модель без инструментов	Агент с инструментами
какие статьи выносить	все восемь	P1, P2, P7, у остальных назван пункт
средняя оценка P2	6,5	8,5
на что опирается 6.5	два верных пункта и выдуманный документ	14 положений из корпуса
утверждение 3 статьи P2	«корректно»	отклонено внешней проверкой
откуда утверждение	неизвестно	пометка и строка протокола

Главное

Модель осталась той же. Изменилось то, что находится вокруг неё: схема данных, граф, правила, внешняя проверка.

Задание 1. Корпус, карта, вопросы

Делаем вживую на новой области

Новая область: правила зачёта

Ведомость, баллы из 25

	зад1	зад2	зад3	зад4	сумма
Аня	22	20	18	21	81
Боря	20	21	22	---	63
Вика	24	23	25	20	**92**
Гоша	23	22	---	---	45
Даша	15	18	20	---	53

Красным: задание сдано позже срока.

Правила, 12 записей

- 3.1: сдано не менее трёх заданий.
- 3.2: каждое задание сдано не позже срока.
- 3.3: сумма баллов не меньше 60.
- 3.4: зачёт ставится при выполнении 3.1--3.3.

Правила вымышленные. К правилам нашего курса они отношения не имеют.

Прогноз

У Вики лучшая сумма баллов в группе. Получит ли она зачёт?

Маршрут задания 1



- **Область** подходит, если в ней есть утверждения, зависимости между ними и хотя бы один проверяемый факт.
- К корпусу прилагаются `AGENTS.md` (инструкции агенту-помощнику) и `diary.md` (журнал работы).
- Всё лежит в шаблоне репозитория. Стартовый набор `starter/` показывает заполненный шаблон.

Блокнот `practice_3_tasks.ipynb`, шаги 1--4.

Как фраза становится строкой корпуса

> «Зачёт выставляется студенту при выполнении требований 3.1, 3.2 и 3.3.» Правила зачёта, §3.4

Четыре вопроса к фразе

1. Что это: раздел, термин, требование?
2. Как коротко назвать?
3. **На что она ссылается?**
4. Где это написано?

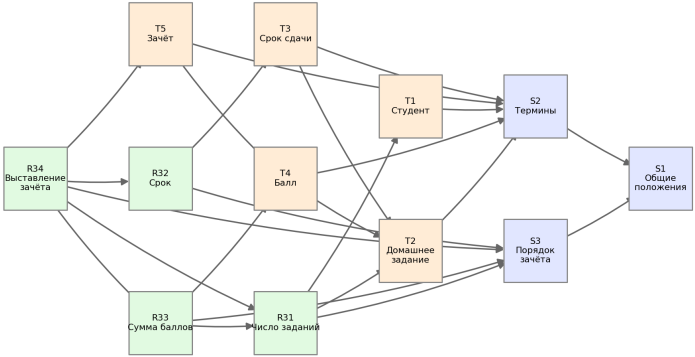
Строка `corpus.csv`

```
id:           R34
тип:          Требование
название:     3.4 Выставление зачёта
формулировка: Зачёт выставляется ...
зависит_от:   S3;T5;R31;R32;R33
источник:     Правила зачёта §3.4
```

Главное

Главный столбец `зависит_от`. В него записываются только прямые ссылки. Цепочки получит граф.

Корпус как граф



12 записей, 23 стрелки. У требования 3.4 записано пять зависимостей. С учётом цепочек оно опирается на 11 положений, то есть на весь остальной корпус.

Пять вопросов исследователя

Тип	Вопрос	Ожидаемый ответ	Источник
(1) факт	Сколько заданий нужно сдать для зачёта?	не менее трёх	R31
(2) зависимость	На какие положения опирается требование 3.4?	на 11 положений	R34 и цепочки
(3) смысл	Есть ли норма об опоздании, если слова «опоздание» в тексте нет?	да, требование 3.2	R32, T3
(4) проверка	Кто из студентов получает зачёт?	Аня и Боря	ведомость, R31--R33

- У каждого вопроса есть ожидаемый ответ и источник. Такой вопрос можно проверить.
- Типы «факт», «зависимость» и «смысл» обязательны.

Что увидели

Вика с суммой 92 зачёт не получает: второе задание сдано позже срока, требование 3.2.

Скрипт подсказывает, что осталось

```
$ python3 scripts/check_hw1.py
== Корпус
  [зачтено] колонки id, тип, название, формулировка, зависит_от, источник — 6 колонок
  [НЕ зачтено] корпус ≥ 30 утверждений — строк: 12
  [зачтено] все id уникальны — повторов нет
  [зачтено] все ссылки в зависит_от существуют — все ссылки найдены
  [зачтено] ≥ 10 строк с непустым зависит_от — строк с зависимостями: 11
== README
  [зачтено] 5 вопросов исследователя заполнены (тип, вопрос, ответ, источник)
  [зачтено] среди вопросов есть типы факт, зависимость, смысл — все три есть
== Файлы проекта
  [зачтено] kg/MAP.md (карта знаний) есть и заполнен
Итого: 15 из 16 пунктов зачтено. Исправьте незачтённые пункты.
```

- Стартовый набор показывает способ. Объём 30 записей набирается на своей области.
- Скрипт запускается перед сдачей. Он же ловит битые ссылки в `зависит_от`.

Пример 5. Онтология и машина вывода

Как перенести правило из кода в данные?

Требование 6.2 переезжает

Пример 3: правило в коде

```
"6.2": not d["своя_организация"].any()
```

Знание о требовании находится в программе. Агент видит только итог: «нет, 6.2».

Пример 5: правило в данных

```
Рецензия(?r), на_статью(?r, ?s),
рецензент(?r, ?p), из_организации(?p, ?o),
организация_авторов(?s, ?o)
  -> нарушает(?s, R62)
```

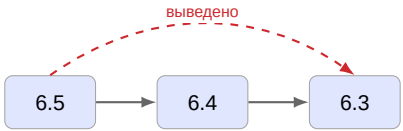
Правило лежит в файле онтологии рядом с фактами. Его исполняет машина вывода.

- **Онтология**: классы, свойства, индивиды, ограничения, правило. Файл `data/review.owl`.
- **Машина вывода**: программа, которая строит следствия из онтологии и проверяет её непротиворечивость.

Блокнот `practice_2_ontology.ipynb`.

Одно слово вместо запроса

Свойство `зависит_от` объявлено транзитивным. Это одно слово в онтологии.



Прогноз

У требования 6.5 записано 4 зависимости. Сколько их будет после запуска машины вывода?

Что увидели

Записано 4, после вывода 14. То же число дал запрос пути в примере 2. Здесь запроса никто не писал.

В задании

Задание 2. Пункт «один выведенный факт»: что записано, что выведено, по какому основанию.

Чего машина вывода не видит

статья	вывод машины	правила над журналом из примера 3
P3	**нарушает R62**	6.2: рецензент из организации авторов
P4	---	6.1: одна рецензия
P5	---	6.6: рецензия на 27-й день
P6	---	6.5: нет ответа авторов
P8	---	6.1: рецензия не сдана

- Машина нашла нарушение 6.2. О статье с одной рецензией она молчит.
- **Открытый мир**: отсутствие записи о второй рецензии не означает, что рецензии нет.

Главное

Машина вывода отвечает на вопрос «что следует из записанного». Вопрос «всё ли записано» решает проверка данных. Это лекция 3.

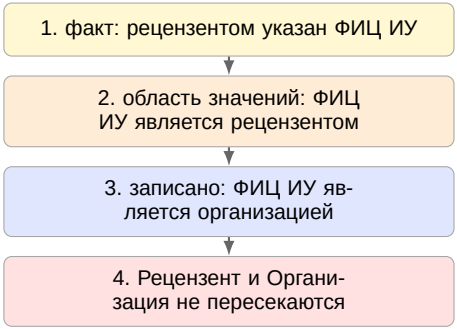
Строка журнала с ошибкой

статья	рецензент
P8	**ФИЦ ИУ**

В поле «рецензент» по ошибке записана организация.

Прогноз

Что произойдёт?
А. Ошибка при записи
Б. Ничего
В. Сообщение о противоречии



Что увидели

Запись принята без возражений. Машина вывода сообщает о противоречии и называет четырёх участников.

Задание 2. Онтология

Делаем вживую на той же области

Шесть решений от карты к онтологии

Решение	Вопрос к себе	В правилах зачёта
классы	какие значения в столбце «тип»? что над ними?	Объект > Положение > Раздел, Термин, Требование. Студент. Сдача
свойства	какие стрелки на карте?	`зависит_от` (транзитивное), `автор`, `балл`
дизъюнктность	у каких классов нет общих объектов?	Положение, Студент, Сдача
ограничения	где фразы «у каждого X есть Y»?	у каждой сдачи есть автор
индивиды	есть ли цепочка из трёх звеньев?	12 положений, два студента, три сдачи
противоречие	что записать в «чужое» поле?	требование в поле «автор»

Главное

Строка корпуса почти всегда индивид. Значение столбца «тип» почти всегда класс.

Запись: классы как классы Python

```
with onto:
    class Объект(Thing): pass
    class Положение(Объект): pass
    class Требование(Положение): pass
    class Студент(Объект): pass
    class Сдача(Объект): pass
    AllDisjoint([Положение, Студент, Сдача])

    class зависит_от(ObjectProperty,
                     TransitiveProperty):
        domain = [Положение]; range = [Положение]
    class автор(ObjectProperty, FunctionalProperty):
        domain = [Сдача]; range = [Студент]

    Сдача.is_a.append(автор.some(Студент))
```

- Класс онтологии записывается как класс Python. Наследование задаёт «подкласс».
- Транзитивность: одно слово.
- `AllDisjoint`: без этой строки противоречие получить нельзя.
- Индивиды создаются циклом по строкам `corpus.csv`.

Проверка фразой: «каждое требование является положением». Звучит верно.

Блокнот `practice_3_tasks.ipynb`, шаг 6. Файл `starter/src/build_ontology.py`.

Выведенный факт и противоречие

Выведенный факт

```
записано:     R31, R32, R33, S3, T5
после вывода: R31, R32, R33, S1, S2, S3,
              T1, T2, T3, T4, T5
выведено:     S1, S2, T1, T2, T3, T4
```

Сохраняйте файл **до** запуска машины вывода.
Иначе выведенное попадёт в файл как записанное.

Противоречие: требование в поле «автор»

```
сдача_ошибка автор R31
автор range Студент
R33 зависит_от R31
зависит_от range Положение
DisjointClasses(Положение Сдача Студент)
```

Ошибка вносится в **свежую копию**. Сдаваемый файл остаётся без ошибки.

В задании

В `kg/ONTOLOGY.md` записывается то, что машина выдала на самом деле: записано, выведено, основание. Внесено, сообщение машины, как исправлено.

Проверка задания 2

```
$ python3 scripts/check_l2.py
== Онтология
  [зачтено] классов ≥ 6 — классов: 7
  [зачтено] иерархия глубже одного уровня — есть цепочка A ▫ B ▫ C
  [зачтено] свойств ≥ 3 — объектных: 2, свойств данных: 1
  [зачтено] ограничений ≥ 2 — ограничений: 2
  [зачтено] объявлена дизъюнктность — записей о дизъюнктности: 2
  [зачтено] индивидов ≥ 5 — индивидов: 17
== Машина вывода
  [зачтено] запись непротиворечива (машина вывода) — противоречий нет
  [зачтено] машина вывода получила новые факты
== Описание и журнал
  [зачтено] в ONTOLOGY.md есть все разделы — разделы на месте
  [зачтено] в AGENTS.md есть словарь классов — найден
Итого: 20 из 20 пунктов зачтено.
```

Подробный разбор задания с образцом на планиметрии: файл `12_КОНСПЕКТ_студента_лекция_2`, раздел 8.

Пример 6. Агент с онтологией

Что делает агент, когда данные испорчены?

Прогон на чистых данных

Вопрос: нарушает ли статья P3 требования положения, и на какие положения опирается нарушенное требование?

```
[1] модель вызвала check_consistency({})
    <- {"непротиворечива": true}
[2] модель вызвала infer({'article': 'P3'})
    <- {"выведено": [{"факт": "P3 нарушает R62 (6.2 Независимость рецензента)",
                      "правило": "правило 6.2 (SWRL)"}]}
[3] модель вызвала graph_deps({'doc_id': 'R62'})
    <- {"положение": "R62", "зависит_от": ["S01 Раздел 1", "S03 Раздел 3", "S06 Раздел 6", ...
[4] ответ модели:
Запись непротиворечива [найдено: check_consistency]. Статья P3 нарушает требование 6.2
[выведено: правило 6.2]. Требование 6.2 опирается на разделы 6, 3, 1 и термины Рецензент, Автор,
Статья, Редакционная коллегия [найдено: graph_deps]. О требованиях 6.1 и 6.6 инструменты
сведений не дали [не подтверждено].
```

- Каждый выведенный факт приходит вместе с правилом.
- Последняя фраза: агент сообщает, о чём у него сведений нет.

Прогон на испорченных данных

Прогноз

Тот же вопрос. В журнале строка с организацией в поле «рецензент». Что ответит агент?

```
[1] модель вызвала check_consistency({})
    <- {"непротиворечива": false, "объяснение": "DisjointClasses(... Организация ... Рецензент ...)
        рец_P9_ошибка рецензент ФИЦ_ИУ  рецензент range Рецензент  ФИЦ_ИУ type Организация"}
[2] ответ модели:
Запись журнала противоречива [противоречие в данных]: индивид ФИЦ_ИУ указан рецензентом
и одновременно является организацией. Выводы о статьях не делаю до исправления строки.
```

Что увидели

Агент останавливается и называет причину. Агент примера 4 на такой строке выдал бы обычный уверенный ответ.

Граница закреплена и в коде: пока запись противоречива, инструменты `infer` и `graph_deps` выводов не выдают.

Вопросы агенту

Вопрос

```
$ python3 ask.py --agent starter
вопрос> Почему Вика не получает зачёт?

$ python3 ask.py --agent l2
вопрос> Нарушает ли статья P1 требования?

$ python3 ask.py --plain "тот же вопрос"
```

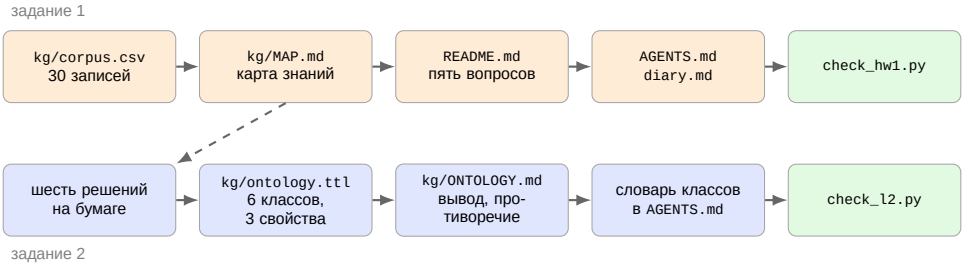
Три агента: задача редактора (`l1`), онтология (`l2`), правила зачёта (`starter`). Ключ `--plain` задаёт вопрос модели без инструментов.

На что смотреть

- Какие инструменты модель вызвала и в каком порядке.
- Есть ли каждое утверждение ответа в результатах инструментов.
- Что агент делает с вопросом, на который инструменты не отвечают.

Проверка агента: вопрос о том, чего в корпусе нет.

Маршрут двух заданий



- Задание 2 строится на корпусе и карте задания 1. Новый материал искать не нужно.
- Оба задания заканчиваются запуском скрипта. Скрипт говорит, что осталось.
- Выполнение заданий на одной странице: файл `ВЫПОЛНЕНИЕ_заданий.md`.

С чего начать

```
# 1. открыть стартовый набор
cd lecture_3/starter

# 2. четыре команды
python3 scripts/check_hw1.py
python3 src/build_ontology.py
python3 scripts/check_l2.py
python3 src/agent_my.py

# 3. заменить kg/corpus.csv своей областью
#    и пройти те же четыре команды
```

Материалы занятия

- три блокнота с сохранёнными результатами.
- стартовый набор `starter/`.
- мини-агент и `ask.py`.
- выполнение заданий на одной странице: `ВЫПОЛНЕНИЕ_заданий.md`.

Без модели всё запускается: инструменты работают по-настоящему, ответ модели берётся из записи. Модель на своей машине: раздел «Установка» в `README.md`.

Что унести

1. Модель без инструментов отвечает уверенно и ошибается там, где ей не дали схемы, связей, правил и проверки.
2. Поиск по словам находит документ. Зависимости находит граф.
3. Проверка состоит из правила, данных со схемой и кода.
4. Агент есть цикл: модель просит, программа исполняет. Проверяют инструменты.
5. Онтология переносит правило из кода в данные. Машина вывода получает новые факты и находит противоречия.
6. Оба задания проходятся по маршруту и заканчиваются запуском скрипта.

Главное

Ответ на вопрос редактора изменился дважды за занятие. Модель при этом ни разу не менялась.

Вопросы

oataeva@frccsc.ru khalov.a@phystech.edu